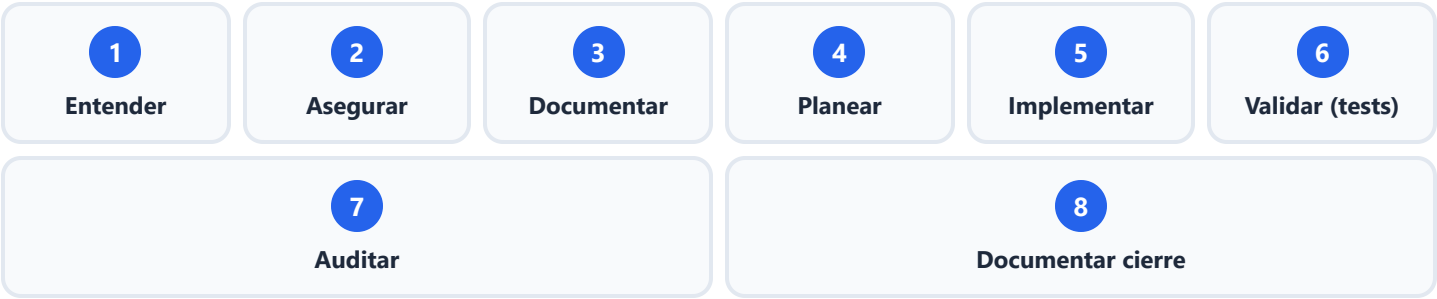


# La Ruta del Proyecto + User Secrets

HackerRank1 (LibraryService API) · Guía de estudio para Ingeniería en Sistemas · Cómo llevamos el proyecto de la base inicial a la versión actual

## PARTE A · LA RUTA DEL PROYECTO

### A1. La ruta en una frase



Ese es el ciclo real de ingeniería de software, y es el mismo patrón que se repite en proyectos profesionales.

### A2. Mapa de la ruta (commits reales)

| Commit  | Fase                                        | Qué se hizo                                                    | Por qué importa                              |
|---------|---------------------------------------------|----------------------------------------------------------------|----------------------------------------------|
| 55cead1 | 0. Base heredada                            | Scaffold del lab ya existente                                  | Entender el terreno antes de tocar nada      |
| bb96648 | 1. Puesta en marcha segura                  | Conexión a Supabase + credenciales fuera de git (User Secrets) | Seguridad decidida <b>antes</b> de programar |
| bd3861b | 2. Documentar el contrato                   | DOCUMENTACION-TECNICA.md (344 líneas) + README.md              | Fija el "qué y cómo" antes del código        |
| cbe5a1b | 3–6. Plan → Implementar → Validar → Limpiar | Endpoints + servicios + arreglo de tests + auditoría           | El corazón del trabajo                       |
| e95aee8 | 7. Cierre                                   | Guía visual de User Secrets                                    | Documentación para futuros lectores          |

### A3. Fase 0 — Leer el terreno

## El punto de partida

El repo llegó con un **scaffold casi completo**: API .NET 8 ( `net8.0` ), **2 proyectos** (API + tests de integración), EF Core 8 + Npgsql contra **Supabase**, JWT ( `AuthController` + `TokenGenerator` ), migración `InitialCreate` y un README que definía el contrato.

Lo primero fue **mapear, no codificar**:

- ¿Qué endpoints faltaban? → `DELETE` en `libraries`; `POST` y `GET` validado en `books`.
- ¿Qué servicios había que completar? → `LibrariesService.Delete` , `BooksService.Add/Update/Delete` .
- ¿Quién define "hecho"? → Los **3 tests de integración**.

**Lección 1:** un buen ingeniero dedica el primer 20% del tiempo a entender la arquitectura y los contratos existentes, no a escribir código inmediato.

## A4. Fase 1 — Seguridad desde el día 1

### Las credenciales no suben a git

Antes de agregar una línea de endpoints, resolvimos lo crítico: **la contraseña de Supabase no podía subirse al repositorio**.

- `appsettings.json` quedó con el placeholder `[SUPABASE-PASSWORD]` (estructura pública, sin secreto).
- La password real vive en **User Secrets** (archivo local fuera del repo, referenciado por `UserSecretsId` en el `.csproj` ).

**Lección 2:** los secretos son una decisión de **arquitectura inicial**, no un parche al final. Es mucho más difícil sanear un secreto ya expuesto en git.

## A5. Fase 2 — Documentar el contrato antes de programar

### Documentación que vive con el código

Se creó `docs/DOCUMENTACION-TECNICA.md` : stack, arquitectura, estructura, bitácora, configuración, comandos y onboarding (sección 7.1). Se actualizó el `README.md` (versión de .NET, `--project` , setup de credenciales).

**Lección 3:** se documenta el **qué se hará** y el **qué ya existe** antes de implementar, para que el código tenga un contrato contra el cual validarse y que otra persona entienda el repo sin adivinar.

## A6. Fase 3 — Plan con un contrato ejecutable

## El test manda

El plan se derivó de leer **primero el test**, no el README: los tests son el contrato que evaluará el instructor.

- **Alcance:** DELETE /api/libraries/{id} y POST / GET /api/libraries/{id}/books (+ servicios).
- **Decisiones de diseño:** DTO BookForm, default Category = string.Empty (columna NOT NULL en Postgres), respuesta **201, 404** si la librería no existe.
- **Contrato:** POST→201/404 · GET→200/404 · DELETE→204/404.

**Lección 4:** en un lab, la "definición de hecho" (Definition of Done) la escriben los tests. Se codifica para que **el test pase**, no para que "se vea bien".

## A7. Fase 4 — Implementar y resolver la cadena de fallos

## El código fue directo; la depuración fue la maestría

- `LibrariesService.Delete` ( `Remove` + `Save` + `true` ).
- `BooksService.Add/Update/Delete` .
- `LibrariesController.Delete` → 404/204.
- `BooksController` reescrito: GET valida librería (404), POST mapea `BookForm` → `Book` y devuelve `CreatedAtAction` (201).

### La cadena de causas raíz:

| # | Síntoma                                                    | Causa raíz                                                                        | Fix                                                                               |
|---|------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1 | <code>MSB1001</code> en <code>dotnet test --project</code> | SDK 10 no acepta <code>--project</code> en <code>dotnet test</code>               | Pasar la ruta del <code>.csproj</code>                                            |
| 2 | <code>MissingMethodException</code>                        | Tests con <b>EF 6.0.0</b> vs API con <b>EF 8.0.2</b>                              | Subir paquetes del test a 8.0.x                                                   |
| 3 | <code>Migrate()</code> rompía SQLite en memoria            | Auto-migrado sobre contexto de tests ( <code>EnsureCreated</code> )               | Guard en <code>Startup</code> : auto-migrar solo si no es SQLite y hay pendientes |
| 4 | <code>Scheme already exists: Bearer</code>                 | <code>ConfigureServices</code> se ejecutaba <b>dos veces</b> (Program + test)     | Guard de idempotencia en <code>ConfigureServices</code>                           |
| 5 | POST books devolvía <b>400</b>                             | <code>Nullable=enable</code> vuelve obligatorio un <code>string</code> no-nulable | <code>BookForm.Category</code> y <code>User.Role</code> → <code>string?</code>    |
| 6 | Tests esperaban books públicos                             | El contrato del lab no exige JWT en books                                         | Quitar <code>[Authorize]</code> de books                                          |

**Lección 5:** nunca se parchea el síntoma. Cada error se resolvió encontrando su causa raíz, que a veces estaba en otra capa (versiones, arranque, serialización, validación).

## A8. Fase 5 — Auditoría y limpieza

## "Funciona" no es igual a "está bien"

Auditoría **en vivo** (no solo compilando): la app arrancó real con `Now listening on: https://localhost:7098` y completó `POST/GET/DELETE` contra Supabase. Resultado: **3/3 tests en verde**.

- `BookForm` → atributos de **System.Text.Json** (antes Newtonsoft, sin uso real).
- Eliminado `DTO/LibraryForm.cs` (muerto).
- `MSTest.TestFramework` y `Newtonsoft.Json` movidos del proyecto de producción al proyecto de tests.
- Borrado el proyecto de tests **huérfano** `IntegrationTest/` (net6 + EF6, fuera de la solución).

**Lección 6:** un proyecto se termina cuando se **audita, se limpia y se documenta**, no cuando el último test pasa.

## A9. Fase 6 — Cierre

### Documentar lo aprendido

Sección 11 del doc técnico (log de implementación) + guía visual de User Secrets. Todo con **commits atómicos** y mensajes semánticos ( `chore:` / `docs:` / `feat:` ), pusheados de forma incremental.

✓ **Resumen:** Entender → Asegurar → Documentar → Planear → Implementar → Validar contra tests → Auditar y limpiar → Cerrar documentando.

## PARTE B · USER SECRETS EXPLICADO

## B1. El problema

Si escribes la contraseña en `appsettings.json`, queda **pública en GitHub** (git guarda el historial). User Secrets es un **cajón secreto que solo existe en tu máquina**.

## B2. El diagrama de capas (lo más importante)

Al arrancar, la configuración se construye apilando capas. **Las de abajo reemplazan a las de arriba**.

1

### appsettings.json

La "cartelera" — estructura con placeholder `[SUPABASE-PASSWORD]`

Base · sube a git

se sobrescribe

2

### appsettings.Development.json

Logging — solo si el entorno es **Development**

Solo Development

se sobrescribe

3

### User Secrets ★

El "cajón" — la password real. **Gana** sobre appsettings.

Password real



se sobrescribe

4

### Variables de entorno

Gana sobre todo (si estuvieran definidas)

Última palabra

✓ `Configuration.GetConnectionString("DefaultConnection")` devuelve la cadena **completa con la password real**.

⚠ Sin User Secrets, la cadena queda con el literal `[SUPABASE-PASSWORD]` y la conexión falla.

## B3. Las tres piezas que se conectan

### Etiqueta (csproj)

HackerRank1/HackerRank1.csproj:7

```
<UserSecretsId>9cbbfeab-...-a68ef6</UserSecretsId>
```

El GUID que conecta el proyecto con su cajón.

✓ **Sí sube a git (no es secreto)**

### Cajón (secrets.json)

`%APPDATA%\Microsoft\UserSecrets\9cbbfeab-...-a68ef6\secrets.json`

Archivo JSON **fuera del proyecto**, en la carpeta del usuario.

✗ **Nunca sube a git**

### Cartelera (appsettings.json)

HackerRank1/appsettings.json:3

Estructura visible con el placeholder `[SUPABASE-PASSWORD]`.

✓ **Sí sube a git (sin secretos)**

## B4. Qué pasa al ejecutar `dotnet run`

1

Se compila; el compilador ve el `UserSecretsId` en el csproj.

2

Detecta que el entorno es **Development** (definido en `launchSettings.json`).

3

Localiza el cajón: `%APPDATA%\Microsoft\UserSecrets\<UserSecretsId>\secrets.json`.

4

Suma las capas: `appsettings.json` → **Development** → **User Secrets** → entorno.

5

User Secrets **reemplaza** el placeholder → conexión real a Supabase. 🎉

⚠ En **Producción** User Secrets **nunca** se cargan. Las credenciales van en variables de entorno o un secreto del hosting.

## B5. Cheat sheet de comandos

Todos se ejecutan **dentro de la carpeta del proyecto** (ahí vive el csproj con el UserSecretsId).

| Acción                 | Comando                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------|
| Guardar / sobrescribir | <code>dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Host=...;Password=..."</code> |
| Ver qué hay guardado   | <code>dotnet user-secrets list</code>                                                              |
| Borrar una clave       | <code>dotnet user-secrets remove "ConnectionStrings:DefaultConnection"</code>                      |
| Desde la raíz          | <code>dotnet user-secrets set ... --project HackerRank1/HackerRank1.csproj</code>                  |

✦ Regla de oro: **estructura en git, secreto en tu máquina**. En otra computadora hay que repetir `dotnet user-secrets set ...` (sección 7.1 del doc técnico).